Підготовка до заняття 9

Evaluations для AI-систем

Про що цей текст

На занятті 8 ми зібрали RAG-бота для документів Northwind Logistics: він відповідає з цитатами і каже «не знайшов», коли відповіді немає. Тепер треба довести цифрами, що він працює, і помітити, коли наступна зміна його зламає. Для цього потрібні метрики. Про кожну з них текст відповідає на ті самі питання: що вона міряє, навіщо, чи рахує її код, чи потрібен LLM-суддя або людина, і яке значення вона дала в нашому прогоні.

Бота ми не змінюємо, змінюється лише промпт. v1 був стартовим промптом заняття 8, v2 з'явився після нього і став еталонною версією, а v3 просить бота «бути кориснішим» і доповнювати відповідь загальною практикою. Усі числа взято з прогону практики: 37 тестових кейсів, 90 розмічених відповідей, `openai/gpt-4o-mini` через OpenRouter з temperature 0 і для бота, і для судді. Читати 40–50 хвилин, а вправи з розділу 11 займуть ще пів години.

План

1. «Здається, працює»
чому один прогін нічого не доводить

2. Що таке eval
тестсет → прогони → метрики → gate

3. Карта метрик
дві родини, три способи рахувати

4. Метрики AI-систем
правильність, формат, стабільність, «не знаю», ціна, корисність

5. Метрики RAG
recall@k, in_context, cited, groundedness

6. Фреймворки
DeepEval, promptfoo, RAGAS

7. Код, суддя чи людина
коли що брати

8. Калібрування судді
чи можна вірити судді

9. Evals у CI/CD
baseline, допуск, регресії

10. Межі цих цифр
37 кейсів, один розмітник

11. Вправи
метрики для інших продуктів, з відповідями

12. Перед заняттям
середовище, TODO 1–7

Що ви вмітимете після заняття

- складати тестовий набір, що ловить реальні помилки;
- обирати метрики під свою задачу і писати для них scoring functions;
- застосовувати LLM-as-a-judge усвідомлено, з калібруванням на розмітці;
- вбудовувати evals у CI/CD і бачити регресії до релізу.

РОЗДІЛ 1

1 «Здається, працює»

Користувачі скаржаться, що бот надто часто каже «не знайшов». Ви пишете промпт v3 («будь кориснішим»), проганяєте 37 тестових питань і отримуєте pass 0.97. У v2 було 0.95, тож здається, що стало краще.

Ми запустили v3 ще двічі з тим самим temperature 0 і обидва рази отримали 0.95. Різницю робило питання q16 про cut-off time для Raben: один раз бот правильно відповів, що послугу припинено (discontinued), двічі сказав, що не знайшов. Temperature 0 зменшує розкид, але провайдер не гарантує однакову відповідь на однаковий запит. Тому один прогін показує лише, що бот *може* відповісти правильно.

У v3 є ще одна проблема, якої не видно ні в pass, ні в повторах: бот почав дописувати до правильних відповідей те, чого немає в документах. Зловить це лише одна метрика з розділу 5.

ГОЛОВНЕ

Відповідь LLM на кейс є **вибіркою**: те саме питання може пройти і впасти. «Я спитав кілька питань, і воно відповіло» не є критерієм. Потрібні фіксований набір кейсів, кілька прогонів і метрики.

Глибше: Hamel Husain: Your AI Product Needs Evals

РОЗДІЛ 2

2 Що таке eval

Eval означає прогін системи на фіксованому наборі кейсів, підрахунок метрик і порівняння з версією, якій ви довіряєте. У практиці він складається з шести кроків:



2.1 Тестсет

Набір має 37 кейсів: 23 із заняття 8 і 14 нових. Кожен тип кейсу відповідає ризику для цього бота: exact 8, paraphrase 7, no_answer 7, ukrainian 4, followup 3, multi_chunk 2, stale 2 (стара версія документа), access 2 (документ, недоступний групі користувача), yes_no 2. На тип треба щонайменше два кейси, бо з одного провалу не зрозуміло, чи бот не вміє цей тип, чи не впорався з одним формулюванням.

```
{"id": "q16", "kind": "stale", "question": "What is the weekday cut-off time for Raben
groupage?",
 "relevant": ["operations/legacy_warehouse_manual_rev7.pdf#42"],   # де лежить відповідь
 "must": [["discontinued"]],                                       # групи варіантів факту
 "reference": "Raben groupage was discontinued by memo QMS-2017-04."}   # пише людина
```

Еталон пише людина, яка знає документи. Якщо його згенерує та сама модель, eval покаже, що бот послідовний, а не що він правий.

Тестсет теж може містити помилки, тому його перевіряють окремо. Команда `--lint` шукає невідомі chunk id, must-факти, яких немає в relevant-чанках, дублікати й нечисленні типи. На нашому наборі вона знайшла q30 («Will the company pay for an upgrade to business class?»): варіанта «No» не може бути в тексті політики, бо це слово відповіді. Змістові помилки lint не бачить. Еталон q13 казав «до 25 кг», а документ каже «25 кг для чоловіків і 15 кг для жінок», і через це суддя валив повні правильні відповіді (розділ 8).

ГОЛОВНЕ

Eval = тестсет, кілька прогонів, метрики, звіт, baseline і gate. Тестсет складається з типів кейсів під ризики, еталон пише людина, а сам набір **теж тестують**: lint знайшов q30, розбіжності з суддею виявили помилку в еталоні q13.

РОЗДІЛ 3

3 Карта метрик

Метрики належать до двох родин. Метрики AI-системи підходять будь-якому LLM-продукту: чи правильно, чи у правильному форматі, чи стабільно, чи чесно каже «не знаю», скільки коштує. Метрики RAG додаються, коли є пошук: чи знайшли потрібний чанк, чи він дійшов до промпту, чи бот на нього послався і чи не дописав від себе.

Друга вісь показує, хто рахує. **Детермінована** метрика рахується кодом: той самий вхід завжди дає те саме число, і це безкоштовно. Якщо треба зрозуміти зміст, рахує LLM-суддя, і число може змінитися між запусками. Людина дає еталони й розмітку, з якими порівнюють і код, і суддю.

	Код (детерміновано)	LLM-суддя	Людина
AI-система	facts, формат і мітки, стабільність, abstain, false abstain, ціна, токени, час	правильність за змістом, корисність за рубрикою	еталонні відповіді, корисність
RAG	recall@k, in_context, cited	groundedness	relevant-чанки в тестсеті, розмітка для калібрування

Правило вибору одне: де можна, рахує код. Суддю беруть для змісту, якого рядок не бачить, і спершу перевіряють на розмітці.

ГОЛОВНЕ

Дві родини метрик (AI-система загалом і RAG) і три способи рахувати (код, суддя, людина). **Код, де можна**, суддя лише для змісту і після калібрування.

РОЗДІЛ 4

4 Метрики AI-систем

4.1 Загальний огляд

Ці метрики не залежать від того, чи є в системі пошук. Змінюється лише те, з чим порівнюють відповідь: у чат-бота це еталонна відповідь, в екстрактора поля JSON, у класифікатора мітка класу, в агента стан після виконання задачі. Через це та сама метрика в різних продуктах рахується по-різному:

Метрика	Чат-бот	Екстракція (JSON)	Класифікація	Агент
Правильність	факти з еталона або суддя	кожне поле = еталон	клас = еталон, точність по класах	кінцевий стан = очікуваний
Формат	схема відповіді, мітки джерел	JSON за схемою, типи полів	мітка з дозволеного списку	виклики інструментів за схемою
Стабільність	той самий набір кілька разів: pass@1, «пройшли щоразу», flaky			
«Не знаю»	«не знайшов» без відповіді в даних	null для поля, якого немає	клас «інше» чи передача людині	зупинка і питання до людини
Ціна і час	$ і токени на прогін, мс на відповідь чи задачу			
Корисність	оцінки користувачів або суддя з рубрикою, відкалібрований на розмітці			

Далі кожна метрика описана в однаковому форматі на прикладі нашого бота. Спершу зведення, значення v2 взято з першого прогону на 37 кейсах.

Метрика	Що міряє	Детермінована?	v2
Правильність: facts	факти з еталона є у відповіді	так · код	0.93
Правильність: judge	зміст збігається з еталоном	ні · суддя	0.97
Формат і мітки	JSON за схемою, мітки джерел існують	так · код	0 невалідних
Стабільність	pass@1 і «пройшли щоразу»	так · код, 3 прогони	0.95 · 0.95
Abstain	«не знайшов» там, де відповіді немає	так · поле found	1.00
False abstain	«не знайшов» там, де відповідь є	так · поле found	0.03
Ціна і час	$ і токени на прогін, мс на відповідь	так · вимір	≈ $0.005
Корисність	чи допомогла відповідь	людина або суддя	не міряли

4.2 Правильність

Що міряє	чи відповідь каже те саме, що еталон
Навіщо	це перше, що хоче знати користувач: бот сказав правду чи ні
Чим рахуємо	facts детермінована (код); judge ні (LLM-суддя порівнює зміст з еталоном)
Як порахувати	facts: бот знайшов відповідь, і з кожної групи `must` у `norm(answer)` є хоча б один варіант. judge: частка вердиктів correct
Приклад	q30: «The company does not reimburse upgrades to business class.» Facts False (жодного варіанта з must), суддя correct. a22 (v1, q08): «...found to be out of tolerance», причина хибна, але слово з must є, тож facts пропустив, і суддя теж. v2: facts 0.93, judge 0.97

`norm()` зводить регістр, пробіли й мінуси до одного вигляду, щоб «−15 °C» з документа збігалося з «-15 °c» у must. Помилки рядка йдуть в обидва боки: він не бачить сенсу «ні» в q30 і пропускає правильне слово в хибному поясненні a22.

4.3 Формат і схема

Що міряє	чи відповідь відповідає схемі і чи мітки джерел існують у контексті
Навіщо	відповідь розбирає код; зламаний JSON чи посилання на неіснуюче джерело ламають продукт ще до питання про зміст
Чим рахуємо	детермінована: валідація схеми і перевірка міток проти виданих джерел
Як порахувати	частка відповідей, що пройшли валідацію; невалідні мітки = мітки поза множиною джерел
Приклад	бот відповідає через structured output з `strict: true`, тож JSON валідний завжди. Невалідних міток 0 у 259 відповідях v1, v2 і v3 (поле `invalid`). На занятті 8 ця перевірка зловила мітки «[S3]» з дужками

4.4 Стабільність

Що міряє	чи бот відповідає однаково добре, якщо спитати знову
Навіщо	один прогін може випадково бути зеленим (розділ 1)
Чим рахуємо	детермінована: код по вже оцінених прогонах; платимо за кожен прогін бота
Як порахувати	для кожного кейсу частка прогонів, де він пройшов. pass@1 = середнє цих часток; «пройшли щоразу» = частка кейсів з часткою 1; flaky = кейси з часткою між 0 і 1
Приклад	v2: прогони 0.95 / 0.95 / 0.95, pass@1 0.95, пройшли щоразу 0.95, flaky немає. v3: 0.97 / 0.95 / 0.95, pass@1 0.955, пройшли щоразу 0.95, flaky q16 (1 з 3)

Назва pass@1 прийшла з оцінювання генерації коду, де pass@k означає, що хоча б одна з k спроб правильна. Для користувача важливіша друга цифра: людина, яка спитає v3 про Raben, отримає одну з двох відповідей.

4.5 Abstain і false abstain

Що міряє	abstain: бот каже «не знайшов», коли відповіді немає. False abstain: каже це, коли відповідь є
Навіщо	вигадана відповідь шкодить, а бот, що на все каже «не знаю», марний. Одну метрику легко покращити коштом другої, тому міряють обидві
Чим рахуємо	детермінована: поле `found` у відповіді
Як порахувати	abstain = частка found=false на кейсах no_answer; false abstain = частка found=false на кейсах з відповіддю
Приклад	abstain 1.00 у v1, v2, v3 (7 кейсів). False abstain: v1 0.10 (q09, q30, q31 з 30), v2 0.03 (лише q09). v1 на так/ні відповідав «не знайшов»

4.6 Ціна, токени, час

Що міряє	скільки коштує прогін у $ і токенах і скільки триває відповідь
Навіщо	краща версія може бути дорожчою чи повільнішою, і це теж рішення про реліз
Чим рахуємо	вимір: токени повертає API у кожній відповіді, час міряє таймер у коді
Як порахувати	сума токенів × ціна моделі; середній (або p95) час на відповідь
Приклад	прогін 37 кейсів: бот ≈ $0.005, суддя ≈ $0.0056, тобто стільки ж, скільки бот. Три прогони v2 коштували $0.0159. Контекст v2 у середньому 459 токенів на питання. Час у цьому прогоні не записували; на занятті 8 v2 відповідав у середньому за 1451 мс

4.7 Корисність

Що міряє	чи відповідь допомогла людині розв'язати задачу
Навіщо	правильна, але незрозуміла чи неповна відповідь теж провал для користувача
Чим рахуємо	не детермінована: оцінки користувачів (кнопки під відповіддю), розмітка або суддя з рубрикою
Як порахувати	частка відповідей, позначених корисними
Приклад	у практиці не міряли: для цього потрібні оцінки користувачів або окремий суддя з калібруванням

ГОЛОВНЕ

Метрики AI-системи однакові для будь-якого LLM-продукту, змінюється лише еталон. Майже всі рахує код. Суддя потрібен для правильності за змістом і корисності, а **abstain і false abstain** міряють лише парою.

Глибше: Chen et al.: pass@k · OpenAI: Evaluation best practices

РОЗДІЛ 5

5 Метрики RAG

RAG ламається в різних місцях: пошук не знайшов потрібний чанк, бюджет токенів його відрізав, модель послалась не туди або дописала від себе. На кожне місце є своя метрика, і разом вони показують, де шукати причину провалу. Значення v2 рахуються на 30 кейсах, де в тестсеті є relevant-чанки (перший прогін).

Метрика	Що міряє	Детермінована?	Де ламається	v2
recall@5	потрібний чанк серед 5 знайдених	так · код	пошук або дані	0.97
in_context	потрібний чанк дійшов до промпту	так · код	бюджет контексту	0.97
cited	потрібний чанк серед процитованих	так · код	генерація	0.97
groundedness	кожне твердження є в цитованих чанках	ні · суддя	генерація	0.97
правильність	як у 4.2, для всієї системи	код і суддя	будь-де	0.93 · 0.97

5.1 recall@k

Що міряє	чи є потрібний чанк серед k знайдених
Навіщо	без потрібного чанка модель не відповість, і жоден промпт не допоможе
Чим рахуємо	детермінована: код порівнює id знайдених чанків з `relevant`, які в тестсет вписала людина
Як порахувати	частка кейсів з відповіддю, де хоча б один relevant-чанк є серед k знайдених
Приклад	v2: 29 з 30, 0.97. Пропущено лише q09 («What is agreement BV-2013-02 about?») в усіх версіях

q09 показує, навіщо метрики по етапах. Виноска «decisions2» відірвалась від свого тексту ще при chunking, тому правильного чанка пошук не знаходить. Метрики відповіді кажуть лише «неправильно», а recall@5 вказує, що лагодити треба дані, а не промпт.

5.2 in_context

Що міряє	чи знайдений чанк потрапив у промпт після ліміту токенів
Навіщо	між пошуком і моделлю стоїть бюджет контексту (1500 токенів у `rag.py`), і великий чанк може не влізти
Чим рахуємо	детермінована: код порівнює id чанків у промпті з `relevant`
Як порахувати	частка кейсів з відповіддю, де relevant-чанк є серед переданих у промпт
Приклад	v2: 29 з 30, 0.97, як і recall@5. Отже, бюджет у нашому прогоні нічого не відрізав

5.3 cited

Що міряє	чи бот послався на правильний чанк
Навіщо	користувач перевіряє відповідь за посиланням; посилання не туди вводить в оману
Чим рахуємо	детермінована: перетин процитованих id з `relevant`; невалідні мітки `rag.py` відкидає ще до оцінки
Як порахувати	частка кейсів з відповіддю, де хоча б один relevant-чанк серед процитованих
Приклад	v2 і v3: 0.97, пропущено q09. v1: 0.87, пропущено q09, q16, q30, q31. Межа: cited не каже, чи текст відповіді справді взято з цього чанка

5.4 Groundedness

Що міряє	чи кожне твердження відповіді є в процитованих чанках (в інших бібліотеках faithfulness)
Навіщо	ловить, коли бот дописує від себе, навіть якщо ключовий факт правильний
Чим рахуємо	не детермінована: суддя ділить відповідь на твердження і для кожного ставить supported; еталон не потрібен
Як порахувати	відповідь grounded, якщо підкріплені всі твердження; метрика = частка grounded серед відповідей, де бот щось знайшов
Приклад	v2 0.97, v3 0.90. v3, q07: «FEFO is a rotation method used primarily for food and pharmaceutical products», а процитовано лише чанк #37 з абревіатурами

Інші непідкріплені твердження v3: у q02 порада «Ensure you provide them with all necessary details about the theft», якої немає в політиці, а в q08 бот пише про drill probe і заморожені товари з чанка #6, хоча процитував лише #7. Суддя правильності всі три відповіді визнав correct, бо ключовий факт на місці.

Буває й навпаки. v1 на q16 відповів «The weekday cut-off time for Raben groupage is 14:00.» Це число справді стоїть у старій таблиці, тож відповідь grounded, але пам'ятка QMS-2017-04 послугу припинила, і відповідь неправильна. Тому правильність і groundedness міряють окремо: перша порівнює з еталоном, друга з джерелами, і groundedness можна рахувати навіть на логах продакшну, де еталонів немає.

```
# джерело: "Hotel, per night: 120 EUR."     відповідь: "120 EUR; hostels are not allowed."
{"claims": [{"claim": "Hotel limit is 120 EUR",  "supported": true},
            {"claim": "Hostels are not allowed", "supported": false}]}   # → not grounded
```

Бібліотеки на кшталт RAGAS мають метрики зі схожими назвами: faithfulness, answer relevancy, context precision, context recall. Ідея та сама: окремо міряти пошук і окремо відповідь, але визначення відрізняються (розділ 6).

ГОЛОВНЕ

RAG додає чотири метрики: знайшли (recall@k), передали (in_context), послались (cited), не вигадали (groundedness). Вони показують, **на якому етапі** зламалось: q09 впав уже на пошуку через chunking. Grounded не означає правильно: 14:00 у v1 спирається на джерело і все одно хибне.

Глибше: Ragas: Available metrics · Es et al.: RAGAS

РОЗДІЛ 6

6 Фреймворки для evals

У практиці всі метрики ми пишемо самі, щоб бачити, як кожна рахується. У робочому проєкті часто беруть готовий фреймворк: у ньому вже є десятки метрик, звіти й запуск у CI. Ось три бібліотеки: DeepEval, promptfoo, найпопулярніший серед інструментів для LLM-застосунків загалом, і RAGAS, найпопулярніший саме для RAG. Зірки на GitHub перевірено 2026-11-06.

	Для чого	Як пишеш тест	Сильна сторона	Зірки	Ліцензія
DeepEval	«pytest для LLM»: RAG, агенти, діалоги, safety	Python: `LLMTestCase` + метрики з порогом	багато готових метрик, `deepeval test run` для CI	19 330	Apache-2.0
promptfoo	порівняння промптів і моделей у матриці, red teaming	YAML: `prompts`, `providers`, `tests` з `assert`	багато детермінованих assert поруч з model-graded	26 899	MIT
RAGAS	метрики RAG	Python: метрики над питанням, контекстами й відповіддю	метрики пошуку, зокрема детермінований recall за id	16 249	Apache-2.0

Langfuse, Opik і Phoenix популярні не менше, але це платформи observability і трейсингу, де evals є частиною, а не бібліотеки для тестів, тому тут їх не розглядаємо. RAGAS оновлюється рідше: останній push у репозиторій був 2026-02-24.

6.1 DeepEval

Кожен кейс описують як `LLMTestCase` з питанням, відповіддю бота, еталоном і контекстом, а кожна метрика повертає число від 0 до 1. Кейс проходить, якщо score ≥ `threshold`, за замовчуванням 0.5. Метрики за замовчуванням працюють як LLM-as-a-judge. Для власного критерію є `GEval`, де критерій пишуть природною мовою, і `DAGMetric` з деревом рішень.

```
test_case = LLMTestCase(
    input="What does FEFO mean?",
    actual_output=answer,                       # відповідь бота
    expected_output="First expired, first out.",
    retrieval_context=cited_chunks,             # тексти процитованих чанків
)
evaluate([test_case], [FaithfulnessMetric(threshold=0.7)])
# у CI тести з assert_test(...) запускає команда нижче
# (імпорти й точні сигнатури: документація DeepEval)
$ deepeval test run test_bot.py
```

6.2 promptfoo

Тест описують у `promptfooconfig.yaml`, а запускають командою `promptfoo eval`. Перевірки бувають детерміновані (`contains`, `regex`, `is-json`, `cost`, `latency`, `is-refusal`...) і model-graded (`llm-rubric`, `factuality`, `context-faithfulness`...). У кожного assert можна задати `threshold` і `weight`.

```
prompts: ["Answer from the handbook: {{question}}"]
providers: [openai:gpt-4o-mini]
tests:
  - vars: {question: "How much can I spend on food per day when I travel for work?"}
    assert:
      - {type: contains, value: "45"}                                  # код
      - {type: llm-rubric, value: "Says 45 EUR per day, no approval needed"}   # суддя
```

6.3 RAGAS

RAGAS має метрики пошуку (Context Precision, Context Recall, Context Entities Recall, Noise Sensitivity) і відповіді (Faithfulness, Response Relevancy, Factual Correctness), а також прості рядкові String Presence і Exact Match. Найкраще різницю між однаковими назвами видно на Context Recall, бо вона існує в трьох варіантах:

Клас	Як рахується	Детермінована?
`ContextRecall`	суддя ділить еталон на твердження; метрика = частка тверджень, підкріплених знайденими контекстами	ні · LLM
`NonLLMContextRecall`	порівняння рядків замість судді	так
`IDBasedContextRecall`	частка `reference_context_ids`, знайдених серед `retrieved_context_ids`	так

Останній варіант і є нашим recall@5 за chunk id. Contextual Recall у DeepEval з тією ж назвою рахує інше: суддя перевіряє, яку частку тверджень з еталона можна приписати знайденому контексту.

6.4 Наші метрики у фреймворках

Наша метрика	Детерм.?	DeepEval	RAGAS	promptfoo
facts (must)	так	немає, через DAG чи власну	String Presence, Exact Match	contains, regex
правильність (суддя)	ні	GEval з expected_output	Factual Correctness	factuality, llm-rubric, g-eval
groundedness	ні	FaithfulnessMetric	Faithfulness	context-faithfulness
recall@5, in_context (за id)	так	Contextual Recall (суддя, інше визначення)	IDBasedContextRecall; ContextRecall (суддя)	context-recall (model-graded)
формат JSON	так	Json Correctness	немає	is-json, contains-json
abstain	так	немає, власна	немає	is-refusal
ціна, час	так	трекінг вартості в evaluate()	немає	cost, latency
gate у CI	так	deepeval test run + threshold	немає, поріг ставите самі	promptfoo eval + threshold
релевантність (у нас не міряли)	ні	AnswerRelevancyMetric	Response Relevancy	answer-relevance

З таблиці видно головну пастку. Назви у фреймворках схожі, але визначення різні: recall за id і recall за твердженнями, який рахує суддя, дадуть різні числа на тому самому прогоні. Тому перед тим, як узяти готову метрику, прочитайте в документації, як саме вона рахується, а model-graded метрики відкалібруйте на своїй розмітці так само, як ми калібрували суддю в розділі 8.

ГОЛОВНЕ

DeepEval (Python, пороги, `deepeval test run`), promptfoo (YAML, детерміновані й model-graded assert) і RAGAS (метрики RAG) дають готові метрики й запуск у CI. **Однакова назва не означає однакове визначення**: читайте, як метрика рахується, і калібруйте суддю на своїх даних.

Глибше: DeepEval: Metrics · DeepEval: Contextual Recall · promptfoo: Assertions · RAGAS: Metrics · RAGAS: Context Recall

РОЗДІЛ 7

7 Код, суддя чи людина

	Що рахує	Коли брати	Ціна і ризик
Код	facts, cited, recall@k, in_context, abstain, стабільність, ціна	відповідь перевіряється рядком, id чи полем	безкоштовно; на нашій розмітці згода 0.98
LLM-суддя	правильність за змістом, так/ні, groundedness	треба зрозуміти зміст; лише після калібрування	коштує стільки ж, скільки бот; сам помиляється
Людина	еталони, relevant-чанки, розмітка для калібрування	треба знати правильну відповідь	найдорожче; без неї ні коду, ні судді нема з чим порівнювати

Код на 90 розмічених відповідях збігся з розміткою в 0.98 випадків і помилився двічі, на q30 і a22 (розділ 4.2). Там, де рядок не бачить сенсу, працює **LLM-as-a-judge**: окремий виклик моделі отримує питання, еталон і відповідь і повертає вердикт через structured output.

```
# system: prompts/judge-v3/correct.txt
# user:   Question / Reference answer / Assistant's answer
class Verdict(BaseModel):
    reason: str                                  # спершу пояснення
    verdict: Literal["correct", "incorrect"]     # потім рішення
```

Еталон іде в промпт, бо внутрішніх правил компанії модель не знає і без еталона оцінювала б за власними уявленнями. Поле `reason` стоїть першим, бо модель генерує токени по черзі: так вердикт пишеться після пояснення і спирається на нього, а не навпаки.

ГОЛОВНЕ

Спершу питайте, чи може перевірити код. Суддю беріть для змісту і лише після калібрування, пам'ятаючи, що він подвоює ціну прогону. Людина потрібна на початку: **без еталонів і розмітки** ні код, ні суддя не мають з чим порівнювати.

Глибше: Zheng et al.: Judging LLM-as-a-Judge · Eugene Yan: LLM-Evaluators

---

РОЗДІЛ 8

8 Калібрування судді

Перша версія промпту судді на q16 помилилась в обидва боки. «I could not find this...» вона назвала correct, бо раз послугу припинено, то й часу нема, а правильне «The Raben groupage service has been discontinued» назвала incorrect, бо «не дано cut-off time». У середньому такі помилки не видно, тому суддю перевіряють на розмітці.

У `data/labels.json` лежать 90 унікальних відповідей v1, v2 і v3. Для кожної розмітник поставив correct, grounded і пояснення; серед них 6 неправильних, 9 не підкріплених і 11 «не знайшов». Калібрування проганяє суддю по цих відповідях і рахує: `agreement` (частка збігів з розміткою), `false_pass` (суддя пропустив неправильну), `false_fail` (суддя завалив правильну) і список розбіжностей. False pass небезпечніший: він завищує метрики і пропускає гіршу версію в реліз, а false fail блокує нормальні зміни.

Промпти судді	код	correct	grounded	Що змінилось
judge-v1	0.98	0.94	0.92	перша версія
judge-v2	0.98	0.88	0.96	grounded: твердження лише з відповіді; correct: «умова, що змінює зміст, означає incorrect»
judge-v3	0.98	0.92	0.96	correct: оцінювати лише ключовий факт, про який питають
judge-v3, еталон q13 виправлено	0.98	0.99	0.94	промпт той самий, виправлено тестсет

Правка judge-v2 покращила grounded з 0.92 до 0.96, але нове правило в correct зробило суддю надто суворим. Він почав валити правильні відповіді з додатковими правдивими деталями: згода впала до

0.88, розбіжностей стало 11, false fail 0.11. Промпт судді такий самий промпт, як у бота, і кожна його правка потребує нового калібрування.

Розбіжності читають усі, бо помилитися міг суддя, розмітник або еталон. У judge-v3 з семи розбіжностей шість були помилкою еталона q13, і після його виправлення згода зросла з 0.92 до 0.99 без зміни промпту. Залишилась a22 з хибною причиною «out of tolerance». Пам'ятайте й про шум: grounded на тому самому промпті дав 0.96 і 0.94 у двох запусках, тож різниця 0.02 нічого не доводить.

ГОЛОВНЕ

Суддю міряють на розмітці: agreement, false pass, false fail і розбіжності. **Правка промпту може погіршити суддю** (judge-v2), а розбіжність може бути виною тестсету (q13: 0.92 → 0.99).

Глибше: Shankar et al.: Who Validates the Validators? · Hamel Husain: LLM-as-a-Judge

---

РОЗДІЛ 9

9 Evals у CI/CD

Eval, який запускають вручну, ловить регресії лише тоді, коли хтось згадав його запустити. Тому його вбудовують у CI. Спершу фіксують baseline, тобто метрики версії, якій довіряють. У нас це v2, збережена командою `--save-baseline` у `baseline.json`.

Функція `gate()` порівнює нову версію з baseline за трьома правилами:

- метрика впала більше ніж на `TOLERANCE = 0.05`: порушення;
- для метрик з `LOWER_IS_BETTER` (у нас `false_abstain`) навпаки: порушення, якщо вона зросла більше ніж на допуск;
- кейс проходив у baseline, а тепер падає: `regression: qNN`.

Будь-яке порушення дає exit code 1, і крок у CI стає червоним. Допуск 0.05 потрібен через шум: на 37 кейсах один кейс змінює метрику приблизно на 0.03, тож допуск пропускає коливання на один кейс і ловить падіння на два. Регресії по кейсах рахують окремо, бо середнє не змінюється, коли один кейс полагодився, а інший зламався.

```
$ python starter/evals.py --prompt v1 --judge --gate
GATE FAILED
  pass: 0.89 < baseline 0.95
  facts: 0.87 < baseline 0.93
  cited: 0.87 < baseline 0.97
  false_abstain: 0.10 > baseline 0.03
  judge: 0.89 < baseline 0.97
  regression: q16
  regression: q31

$ python starter/evals.py --prompt v3 --repeat 3 --judge --gate
GATE FAILED
  grounded: 0.90 < baseline 0.97
```

З v1 усе очевидно. Цікавіша v3: pass 0.97 проти 0.95, facts 0.97 проти 0.93, суддя правильності 0.97, як і в v2. Без groundedness вона пішла б у реліз як покращення. Зупинила її одна метрика, grounded

0.90 при baseline 0.97: доповнення «з загальної практики», про які просив промпт, виявились твердженнями, яких немає в документах.

```
on:
  pull_request:
    paths: ["prompts/**", "data/testset.json", "rag.py", "retrieval.py", "baseline.json"]
steps:
  - run: python starter/check.py                 # офлайн, без ключа і витрат
  - run: python starter/evals.py --lint          # тестсет без механічних помилок
  - run: python starter/evals.py --judge --gate  # з секретом OPENROUTER_API_KEY
  - uses: actions/upload-artifact@v4             # звіт, навіть при червоному gate
```

`ci/evals.yml` запускається на кожен pull request, що змінює промпти, тестсет, код бота чи baseline, тож заміна моделі в `rag.py` теж проходить через gate. Кроки йдуть від дешевого до дорогого. Workflow написаний, але в GitHub Actions ми його не запускали. Baseline оновлюють свідомо, коли нову версію прийняли, і зміна `baseline.json` видна в diff поруч з промптом.

ГОЛОВНЕ

Gate порівнює кожну зміну з baseline: падіння більше ніж на 0.05 (для false abstain ріст), регресії по кейсах, exit code 1. **v3 краща за pass і facts, але gate зупинив її через grounded 0.90 < 0.97.**

Глибше: GitHub: Workflow syntax · promptfoo: CI/CD

РОЗДІЛ 10

10 Межі цих цифр

Межа	Що це означає
37 кейсів	1 кейс ≈ 0.03 метрики, допуск 0.05 ≈ 1–2 кейси. Різниця 0.97 і 0.95 означає один кейс
Один розмітник	90 відповідей розмітив один розмітник, і це була модель Claude, а не людина. Згоду двох незалежних розмітників не міряли
Суддя = модель бота	`gpt-4o-mini` оцінює `gpt-4o-mini` і може віддавати перевагу власному стилю. Сильнішим суддею не перевіряли
CI на папері	`ci/evals.yml` написаний, але в GitHub Actions не запускався

Тому поруч з кожною метрикою пишіть, на скількох кейсах її пораховано і хто розмічав еталон. У власному проєкті додавайте кейси насамперед у найризиковіші типи і розмічайте хоча б частину відповідей двома людьми.

ГОЛОВНЕ

Ці цифри показують **метод**, а не точну якість бота: різниця в один кейс на 37 нічого не доводить.

Глибше: Anthropic: A statistical approach to model evaluations

РОЗДІЛ 11

11 Вправи

> Вправи не прив'язані до нашого бота. Продукти і числа в них умовні, вигадані для тренування, а не взяті з прогону.

1. *Метрики під продукт.* Для кожного продукту оберіть 4–5 метрик і для кожної скажіть, чи вона детермінована і чому: (а) чат-бот підтримки над статтями help center; (б) екстрактор JSON з рахунків (номер, дата, сума, IBAN постачальника); (в) класифікатор тікетів підтримки за 8 категоріями.
2. *Код, суддя чи людина.* Хто має рахувати кожну перевірку: сума в JSON збігається з еталоном; відповідь ввічлива; бот процитував статтю, яка існує; відповідь не суперечить процитованій статті; категорія тікета дорівнює еталонній; клієнт отримав допомогу; JSON проходить схему; 95-й перцентиль затримки; еталонна відповідь на нове питання?
3. *Стабільність.* Три прогони, ✓ означає «пройшов». Порахуйте pass@1, «пройшли щоразу» і назвіть flaky-кейси.

Кейс	A	B	C	D
Прогони 1, 2, 3	✓ ✓ ✓	✓ ✗ ✓	✗ ✗ ✗	✓ ✓ ✗

4. *Тестсет для нового домену.* Бот відповідає на питання про відпустки за HR-політикою компанії. Складіть 5 кейсів різних типів, серед них один no_answer. Для кожного вкажіть тип, питання, must-факти і еталон. Який кейс ловить застарілу версію політики?
5. *Чи вірити судді.* Два судді на 100 розмічених відповідях (умовні числа). Суддя A: agreement 0.93, false pass 0.06, false fail 0.01. Суддя B: agreement 0.91, false pass 0.01, false fail 0.08. Котрого ставити в gate і що зробити перед цим?
6. *Правила gate.* Класифікатор тікетів, тестсет 50 кейсів. Які метрики покласти в baseline, у якому напрямку кожна «гірша» і який допуск обрати? Що gate має робити з окремими кейсами?
7. *Готова метрика чи своя перевірка.* Ви хочете перевіряти, що RAG-бот знайшов потрібний документ, і в тестсеті для кожного кейсу є id правильних чанків. Фреймворк пропонує метрику context recall, яку рахує LLM-суддя за твердженнями з еталона. Що ви візьмете для gate у CI і чому? Коли готова метрика з суддею все ж доречна?

12 Перед заняттям

> На занятті працюємо з кодом з репозиторію курсу. Підготуйте середовище й ключ заздалегідь, а TODO 1–3 зробіть удома.

```
# перший раз: git clone https://github.com/andbilous/ai-engineeering-2
cd ai-engineeering-2 && git pull
cd lesson-09-evals
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# модель embeddings з занять 7–8 (один раз, у папку .models)
python -c "from fastembed import TextEmbedding; \
TextEmbedding('sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2',
cache_dir='.models')"

cp .env.example .env                        # і вставте свій OPENROUTER_API_KEY
python -c "import tiktoken; tiktoken.get_encoding('o200k_base')"   # словник токенізатора,
один раз
```

Бот і суддя працюють на `openai/gpt-4o-mini` через OpenRouter. Прогін бота на 37 кейсах коштує близько півцента, суддя ще стільки ж. Без ключа працюють перевірки 1–7 з фейковою моделлю, жива перевірка 8 пропускається. Python беріть 3.12, як у CI-workflow.

Файл	Що це
`rag.py`, `retrieval.py`	готовий RAG із заняття 8 і hybrid search із заняття 7; змінювати не треба
`data/testset.json`, `data/labels.json`	37 кейсів дев'яти типів; 90 розмічених відповідей для калібрування
`prompts/`	промпти бота v1–v3, промпти судді judge-v1...v3, `CHANGELOG.md`
`baseline.json`	метрики v2, з якими порівнює gate
`starter/evals.py`, `starter/check.py`	eval-пайплайн з TODO 1–7; 8 перевірок: 7 офлайн і одна з реальним суддею
`ci/evals.yml`	GitHub Actions: check.py → --lint → --judge --gate, звіт як artifact

TODO: що, навіщо і як побачити результат

Перед кожним TODO в коді є вступ (Goal, Why, Done when), таблиця нижче збирає те саме. Пишіть код з AI-асистентом, але кожен рядок маєте вміти пояснити: на занятті хтось із вас покаже свій розв'язок.

TODO	Що зробити	Навіщо	Результат
ВДОМА			
1	`lint()`: невідомі chunk id, must-факт поза relevant-чанками, no_answer з must/relevant, дублікати, тип з <2 кейсами	перевірити тести, перш ніж вірити їхнім числам (2.1)	перевірка 1; `--lint` знаходить q30
2	`score()`: facts, cited, abstain, pass через `norm()`	детерміновані метрики там, де вони працюють (4.2, 5.3)	перевірка 2; прогін v2 дає pass близько 0.95
3	`stability()`: частки по кейсах, pass_at_1, pass_all, flaky	один зелений прогін нічого не доводить (1, 4.4)	перевірка 3; `--repeat 3` друкує flaky
НА ЗАНЯТТІ			
4	`judge()`: `correct.txt` як system, питання + еталон + відповідь, `Verdict`	правильність за змістом, де рядок не бачить сенсу (7)	перевірка 4; q30 у v2 отримує correct
5	`calibrate()`: agreement, false_pass, false_fail, disagreements	перевірити суддю, перш ніж йому вірити (8)	перевірка 5; `--calibrate` друкує таблицю й розбіжності
6	`grounded()`: твердження проти процитованих чанків, `(None, [])` для «не знайшов»	зловити відповіді, що кажуть більше за джерела (5.4)	перевірка 6; у звіті v3 видно непідкріплені твердження
7	`gate()`: падіння більше за TOLERANCE, ріст для LOWER_IS_BETTER, `regression: qNN`	зупинити гіршу версію до релізу (9)	перевірка 7; v3 отримує GATE FAILED через grounded

```
python starter/check.py                  # перевірки 1–8
python starter/evals.py --lint           # після TODO 1: "37 cases, 1 problem(s)", проблема
q30
python starter/evals.py                  # після TODO 2: прогін v2, звіт output/report-v2.md
python starter/evals.py --repeat 3       # після TODO 3: pass_at_1, pass_all, flaky
```

Без змін у коді `check.py` покаже TODO навпроти кожної перевірки, після TODO 1–3 перевірки 1–3 мають бути PASS. Числа прогону у вас можуть трохи відрізнятися, і на занятті ми розберемо, наскільки цьому можна вірити.